Informe de solución

Caso 2: La bolsa de empleo

ICC-223 Bases de Datos

Presentación

Este informe organiza la solución solicitada para la base de datos **Bolsa_Empleo**. El caso pide crear cuatro elementos concretos: una vista, un procedimiento almacenado, una función y varios triggers de auditoría sobre la tabla **Empresa**.

Motor asumido: Microsoft SQL Server (T-SQL)
Entregable preparado: informe en PDF + script SQL listo para ejecutar.

1. Lectura del caso y análisis inicial

El caso se basa en la base de datos de un sitio web de bolsa de empleos y pide lo siguiente:

1. Una vista con la cantidad de ofertas por semana que ha publicado cada empresa por ciudad.
2. Un procedimiento almacenado que devuelva la cantidad total de ofertas publicadas por mes en un rango de fechas.
3. Una función que valide si la cédula de una persona corresponde a una cédula dominicana.
4. Triggers que alimenten una tabla llamada **auditoria_empresa** cuando se modifique la tabla **Empresa**.

La estructura visible del caso muestra como tablas más relevantes **Oferta_Empresa**, **Empresa**, **Ciudad**, **Persona** y varias tablas de catálogos.

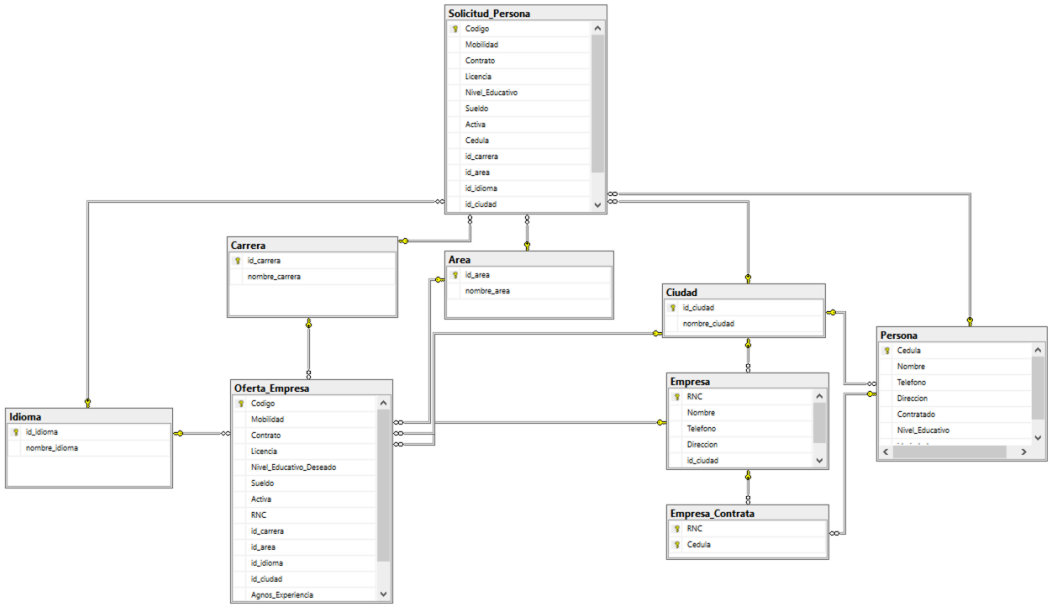


Figura 1: Diagrama suministrado en el caso.

Supuesto necesario

Para poder calcular cantidades **por semana** y **por mes** hace falta un atributo de fecha en **Oferta_Empresa**. En la parte visible del diagrama no aparece ese campo, por lo que en esta solución se asume una columna llamada **fecha_publicacion**.

Si la copia local no la tiene, el script la agrega. Ese supuesto es razonable porque sin una fecha no es posible cumplir los requerimientos 1 y 2 de forma ejecutable.

También se asumió que la ciudad a usar en el reporte semanal es la ciudad de la oferta (`Oferta_Empresa.id_ciudad`) y no la ciudad administrativa de la empresa, porque el enunciado habla de ofertas publicadas por ciudad.

Objetos entregados

Tipo	Nombre	Finalidad
Vista	`vw_OfertasPorSemanaEmpresaCiudad`	Muestra la cantidad de ofertas publicadas por semana, empresa y ciudad.
Procedimiento almacenado	`sp_TotalOfertasPorMes`	Devuelve el total mensual de ofertas en un rango de fechas.
Función	`fn_ValidarCedulaDominicana`	Valida el formato y el dígito verificador usando Luhn.
Tabla de auditoría	`auditoria_empresa`	Guarda la huella de inserciones, actualizaciones y eliminaciones en `Empresa`.
Triggers	`trg_empresa_insert`, `trg_empresa_update`, `trg_empresa_delete`	Registran automáticamente cada cambio realizado sobre `Empresa`.

2. Preparación previa

Antes de crear la vista y el procedimiento, se prepara la tabla **Oferta_Empresa** para asegurar que exista la columna de fecha.

```
USE Bolsa_Empleo;
GO

IF COL_LENGTH('dbo.Oferta_Empresa', 'fecha_publicacion') IS NULL
BEGIN
    ALTER TABLE dbo.Oferta_Empresa
    ADD fecha_publicacion DATE NULL;
END;
GO

UPDATE dbo.Oferta_Empresa
SET fecha_publicacion = ISNULL(fecha_publicacion, CAST(GETDATE() AS DATE));
GO
```

Este bloque permite que la solución funcione incluso si la base copiada localmente no trae el campo de fecha.

3. Vista solicitada

3.1 Objetivo

La vista debe mostrar cuántas ofertas ha publicado cada empresa por ciudad y por semana. Para eso se agrupa por empresa, ciudad, año y semana ISO. El uso del año evita mezclar semanas iguales de años distintos.

3.2 Código

```
CREATE OR ALTER VIEW dbo.vw_OfertasPorSemanaEmpresaCiudad
AS
SELECT
    e.RNC,
    e.Nombre AS nombre_empresa,
```

```
    c.id_ciudad,
    c.nombre_ciudad,
    YEAR(oe.fecha_publicacion) AS anio,
    DATEPART(ISO_WEEK, oe.fecha_publicacion) AS semana,
    COUNT(*) AS cantidad_ofertas
FROM dbo.Oferta_Empresa AS oe
INNER JOIN dbo.Empresa AS e
    ON oe.RNC = e.RNC
INNER JOIN dbo.Ciudad AS c
    ON oe.id_ciudad = c.id_ciudad
WHERE oe.fecha_publicacion IS NOT NULL
GROUP BY
    e.RNC,
    e.Nombre,
    c.id_ciudad,
    c.nombre_ciudad,
    YEAR(oe.fecha_publicacion),
    DATEPART(ISO_WEEK, oe.fecha_publicacion);
GO
```

3.3 Consulta de prueba

```
SELECT *
FROM dbo.vw_OfertasPorSemanaEmpresaCiudad
ORDER BY anio, semana, nombre_empresa, nombre_ciudad;
GO
```

Esta vista puede usarse directamente en reportes, paneles o pantallas de consulta.

4. Procedimiento almacenado

4.1 Objetivo

El procedimiento devuelve el total de ofertas publicadas por mes dentro de un rango de fechas especificado por el usuario. Se validan las fechas para evitar errores básicos de captura.

4.2 Código

```
CREATE OR ALTER PROCEDURE dbo.sp_TotalOfertasPorMes
    @fecha_inicio DATE,
    @fecha_fin DATE
AS
BEGIN
    SET NOCOUNT ON;

    IF @fecha_inicio IS NULL OR @fecha_fin IS NULL
    BEGIN
        RAISERROR('Debe indicar una fecha de inicio y una fecha fin.', 16, 1);
        RETURN;
    END;

    IF @fecha_inicio > @fecha_fin
    BEGIN
        RAISERROR('La fecha de inicio no puede ser mayor que la fecha fin.', 16, 1);
        RETURN;
    END;
```

```
    SELECT
        YEAR(fecha_publicacion) AS anio,
        MONTH(fecha_publicacion) AS numero_mes,
        DATENAME(MONTH, fecha_publicacion) AS nombre_mes,
        COUNT(*) AS total_ofertas
    FROM dbo.Oferta_Empresa
    WHERE fecha_publicacion >= @fecha_inicio
      AND fecha_publicacion < DATEADD(DAY, 1, @fecha_fin)
    GROUP BY
        YEAR(fecha_publicacion),
        MONTH(fecha_publicacion),
        DATENAME(MONTH, fecha_publicacion)
    ORDER BY
        anio,
        numero_mes;
END;
GO
```

4.3 Ejecución de ejemplo

```
EXEC dbo.sp_TotalOfertasPorMes '2025-01-01', '2025-12-31';
GO
```

La aplicación solo tendría que enviar dos parámetros y recibe el resumen mensual listo para mostrar.

5. Función de validación de cédula

5.1 Criterio aplicado

La función recibe una cédula con o sin guiones, limpia el valor, verifica que tenga exactamente 11 dígitos y luego calcula el dígito verificador usando la lógica de Luhn sobre los primeros 10 dígitos. Devuelve **1** si la cédula es válida y **0** si es inválida.

5.2 Código

```
CREATE OR ALTER FUNCTION dbo.fn_ValidarCedulaDominicana (@cedula VARCHAR(20))
RETURNS BIT
AS
BEGIN
    DECLARE @limpia VARCHAR(20);
    DECLARE @i INT;
    DECLARE @posicion INT;
    DECLARE @digito INT;
    DECLARE @valor INT;
    DECLARE @suma INT;
    DECLARE @digito_control INT;

    SET @limpia = REPLACE(REPLACE(REPLACE(ISNULL(@cedula, ''), '-', ''), ' ', ''), '.', '');

    IF LEN(@limpia) <> 11 OR @limpia LIKE '%[^0-9]%'
        RETURN 0;

    SET @i = 10;
    SET @posicion = 1;
    SET @suma = 0;

    WHILE @i >= 1
```

```
    BEGIN
        SET @digito = CONVERT(INT, SUBSTRING(@limpia, @i, 1));

        IF @posicion % 2 = 1
            SET @valor = @digito * 2;
        ELSE
            SET @valor = @digito;

        IF @valor > 9
            SET @valor = @valor - 9;

        SET @suma = @suma + @valor;
        SET @i = @i - 1;
        SET @posicion = @posicion + 1;
    END;

    SET @digito_control = (10 - (@suma % 10)) % 10;

    IF @digito_control = CONVERT(INT, RIGHT(@limpia, 1))
        RETURN 1;

    RETURN 0;
END;
GO
```

5.3 Pruebas

```
SELECT dbo.fn_ValidarCedulaDominicana('001-1391820-5') AS cedula_valida;
SELECT dbo.fn_ValidarCedulaDominicana('001-1391820-4') AS cedula_invalida;
GO
```

6. Auditoría de la tabla Empresa

6.1 Tabla de auditoría

Se crea una tabla llamada **auditoria_empresa** para registrar qué pasó, cuándo pasó, quién ejecutó la acción y cuáles eran los valores anteriores y nuevos.

```
IF OBJECT_ID('dbo.auditoria_empresa', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.auditoria_empresa
    (
        id_auditoria INT IDENTITY(1,1) PRIMARY KEY,
        tipo_accion VARCHAR(10) NOT NULL,
        fecha_evento DATETIME NOT NULL DEFAULT GETDATE(),
        usuario_bd SYSNAME NOT NULL DEFAULT SUSER_SNAME(),
        RNC VARCHAR(20) NULL,
        nombre_anterior VARCHAR(150) NULL,
        nombre_nuevo VARCHAR(150) NULL,
        telefono_anterior VARCHAR(30) NULL,
        telefono_nuevo VARCHAR(30) NULL,
        direccion_anterior VARCHAR(200) NULL,
        direccion_nueva VARCHAR(200) NULL,
        id_ciudad_anterior INT NULL,
        id_ciudad_nueva INT NULL
    );
END;
```

```
GO
```

6.2 Triggers

Para que la auditoría sea clara se crean tres triggers: uno para inserciones, uno para actualizaciones y uno para eliminaciones.

```
CREATE OR ALTER TRIGGER dbo.trg_empresa_insert
ON dbo.Empresa
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.auditoria_empresa
    (
        tipo_accion, fecha_evento, usuario_bd, RNC,
        nombre_nuevo, telefono_nuevo, direccion_nueva, id_ciudad_nueva
    )
    SELECT
        'INSERT', GETDATE(), SUSER_SNAME(), i.RNC,
        i.Nombre, i.Telefono, i.Direccion, i.id_ciudad
    FROM inserted AS i;
END;
GO

CREATE OR ALTER TRIGGER dbo.trg_empresa_update
ON dbo.Empresa
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.auditoria_empresa
    (
        tipo_accion, fecha_evento, usuario_bd, RNC,
        nombre_anterior, nombre_nuevo,
        telefono_anterior, telefono_nuevo,
        direccion_anterior, direccion_nueva,
        id_ciudad_anterior, id_ciudad_nueva
    )
    SELECT
        'UPDATE', GETDATE(), SUSER_SNAME(), i.RNC,
        d.Nombre, i.Nombre,
        d.Telefono, i.Telefono,
        d.Direccion, i.Direccion,
        d.id_ciudad, i.id_ciudad
    FROM inserted AS i
    INNER JOIN deleted AS d
        ON i.RNC = d.RNC;
END;
GO

CREATE OR ALTER TRIGGER dbo.trg_empresa_delete
ON dbo.Empresa
AFTER DELETE
AS
BEGIN
```

```
    SET NOCOUNT ON;

    INSERT INTO dbo.auditoria_empresa
    (
        tipo_accion, fecha_evento, usuario_bd, RNC,
        nombre_anterior, telefono_anterior, direccion_anterior, id_ciudad_anterior
    )
    SELECT
        'DELETE', GETDATE(), SUSER_SNAME(), d.RNC,
        d.Nombre, d.Telefono, d.Direccion, d.id_ciudad
    FROM deleted AS d;
END;
GO
```

6.3 Verificación

```
SELECT *
FROM dbo.auditoria_empresa
ORDER BY id_auditoria DESC;
GO
```

Con esto, cada cambio importante en la tabla **Empresa** deja evidencia automática sin depender de que el usuario lo haga manualmente.

7. Conclusión

La solución propuesta cumple los cuatro puntos del caso de manera directa y ejecutable en SQL Server:

- la vista resuelve la necesidad de seguimiento semanal por empresa y ciudad;
- el procedimiento resume las ofertas por mes dentro del rango indicado;
- la función valida cédulas dominicanas antes de almacenarlas o procesarlas;
- y la tabla de auditoría con sus triggers deja trazabilidad de los cambios realizados sobre **Empresa**.

El único supuesto adicional fue incorporar `fecha_publicacion` en **Oferta_Empresa** cuando esa columna no estuviera disponible en la copia local. Ese ajuste es coherente con el requerimiento y permite que la solución funcione de principio a fin.

Anexo: además del informe, se generó un archivo `.sql` con todo el script listo para ejecutar.